

串連系統 v2.1

完整技術參考手冊 · Complete Technical Reference

管理層決策輔助系統 · 投資公司應用

本手冊完整涵蓋串連系統 v2.1 的所有技術細節：核心演算法（25 項）、資料結構、常數定義、UI 元件、同步機制、權限模型、Bug 修復記錄、設計限制與已知議題。適合 (a) 期末口試 cheatsheet (b) 教師深度檢閱 (c) 新進工程師完整交接 (d) 論文方法章節附錄。本文件不漏任何技術細節。

專案名稱	串連系統 v2.0 (Chuanlien System)
應用場景	20 – 50 人規模投資公司的管理層決策輔助
前端技術	React 19 + TypeScript 5 + Vite 6 + Tailwind v4 + Framer Motion 12 + Recharts 2
後端服務	Firebase Authentication + Cloud Firestore
演算法總數	25 項（22 核心 + 3 工具機制）
UI 模組數	11 個主要頁面
資料模型	10 個 collection (reports, handoffs, decisions, blockers, history, employees, departments, users, meetingHistory, ...)
程式碼行數	約 8,500 行 TypeScript + JSX
作者	資管導論 第 13 組
文件版本	v2.1 完整版 · 2026-05

目錄

第一部 系統概觀

1. 系統總覽與技術棧
2. 檔案結構與模組關係圖
3. 資料模型與型別定義

第二部 演算法詳解 (25 項)

4. ① 資訊檢索 (6 項) — BM25F、IDF、n-gram、Substring、同義詞、Cosine
5. ② 統計分析 (3 項) — Empirical Percentile、Gini、敘述統計
6. ③ 時間序列 (4 項) — Exp Decay、asOf Snapshot、Local Minima、Weekly Series
7. ④ 加權評分 (5 項) — Load Score、ORI、Health 6D、Decision Impact、Leader Score
8. ⑤ 圖論網絡 (3 項) — Adjacency Matrix、Force-directed、Asymmetric Detection
9. ⑥ 狀態判定 (2 項) — Decision Status Helpers、Risk/Load/Health Level
10. ⑦ 預測模擬 (2 項) — What-if Simulation、Smart Suggestion
11. ⑧ 工具機制 (3 項) — Optimistic Sync、SEED Protection、NaN Guards

第三部 常數與配置

12. 全域常數定義表
13. SEED 資料結構
14. 色彩系統與 UI 等級配色

第四部 UI 與互動

15. 11 個主要頁面與功能矩陣
16. 權限模型與角色

17. 通知中心與事件流

第五
部 工
程實
踐

18. Bug 修復記錄 (v2.0 → v2.1, 共 9 件)

19. 已知設計限制與未來改進方向

20. 演算法選用對照與設計哲學

附錄

A. 演算法快速索引 (25 項對應檔案)

B. 公式速查表

C. FAQ — 教授可能問的 12 個問題

1. 系統總覽與技術棧

串連系統 v2.1

是面向中小型投資公司管理層的決策輔助平台。系統的設計核心並非「資料儲存」，而是「演算法層」——在原始資料之上，用統計、資訊檢索、圖論、時間序列、加權評分等方法，把瑣碎事件融合成「組織儀表板」與「決策閉環」（Plan → Decide → Track → Learn）。

1.1 技術棧

前端框架	React 19 (含 hooks: useState, useMemo, useEffect, useCallback)
語言	TypeScript 5 (strict mode, ESM)
打包工具	Vite 6 (HMR, ESBuild, code splitting)
CSS	Tailwind CSS v4 (utility-first, JIT)
動畫	Framer Motion 12 (motion/react, AnimatePresence)
圖表	Recharts 2 (LineChart, BarChart, RadarChart, PieChart)
Icon	Lucide React (TreeShakable SVG icons)
認證	Firebase Authentication (Email/Password)
資料庫	Cloud Firestore (NoSQL document DB)
部署	Vercel (自動 CI/CD from GitHub main branch)
版控	Git + GitHub (Peter2594/chuanlien-system-v2)

1.2 設計原則

- (1) 用恰當的演算法解決恰當規模的問題 — 50 人公司、200 筆歷史案，BM25F 比 LLM Embedding 更可靠。
- (2) 可解釋性優先 — 每個分數都能逐項拆解，管理層能追溯來源。
- (3) 機密敏感 — 全部運算在前端 + 自有 Firebase 跑，零外部 API call。
- (4) 跨頁面一致性 — 同一個指標在不同頁面用同一個分析器，避免「兩邊數字對不上」。
- (5) 互動即時 — 篩選、模擬、切換週次都是 $O(N \log N)$ 級別，瀏覽器毫秒級回應。

3. 資料模型與型別定義

3.1 核心型別 (src/lib/types.ts)

Report	週報	id, dept, week, author, submittedAt, cases, blockers, needHelp, nextWeek, keywords[]
Handoff	案件交接	id, from, to, caseId, title, background, progress, todo, attachments[], status, sender, receiver, createdAt, hoursOverdue?
Decision	決策	id, title, content, decidedBy, decidedAt, dueDate, assignedDept, status, linkedCases?[], notes?, completedAt?
Blocker	卡點	id, title, description, dept, owner, category, status, createdAt, updatedAt, weekId, caseId, relatedDepartments?[], daysToResolve?
HistoryCase	歷史案件	id, title, date, tags[], summary, owner, handoffs, outcome, detail: { background, process, valuation, keyInsights[], result, lessons }
Employee	員工	name, dept, role
Department	部門	id, name, shortName?, active, order?
SystemUser	系統使用者	id, email, name, role (admin/manager/member), dept?, active?
MeetingHistory	歷史會議	id, title, schedule, audience, icon, archivedAt, archivedBy, agendaSnapshot[], textSnapshot

3.2 衍生型別

EmployeeLoad	Employee + { loadScore, percentile, level, timeWeightedCases, blockerLoad, mentionsWeighted, handoffLoad, ... }
HealthSnapshot	{ blockerHealth, decisionTimeliness, handoffSmoothness, loadBalance, crossDept, reportQuality, overall, weekISO, events[] }
DecisionImpact	{ decision, before, after, deltaOverall, deltaByDimension, verdict, score, insufficient?, daysSinceCompleted? }
LeaderScore	{ decidedBy, totalDecisions, completedDecisions, avgImpactScore, positiveCount, neutralCount, negativeCount }
DeptNetwork	{ matrix, edges[], stats, depts[] } — 部門互動圖
ORI	{ index, drivers: [{ code: HCC/DL/BT/CDC, value, weight }] }

4. ① 資訊檢索演算法 (6 項)

4.1 BM25F (Okapi BM25 with Field Weighting) — 主搜尋演算法

用途：歷史案件搜尋、智能推薦。Elasticsearch / Lucene 同款。

```
score(q, d) =  $\sum_{t \in q} \text{idf}(t) \times \sum_f \text{field\_weight}(f) \times \text{tf\_norm}(t, d, f)$ 
```

```
tf_norm(t, d, f) =  $\text{tf}(t, d, f) \times (k_1 + 1)$ 
```

```
tf(t, d, f) =  $\frac{\text{count}(t, d, f)}{\text{tf}(t, d, f) + k_1 \times (1 - b + b \times \text{len}(d, f) / \text{avgLen}(f))}$ 
```

```
k1 = 1.5 (TF 1.2-2.0)
```

```
b = 0.75 (0= 1=)
```

欄位權重表：title=5.0、tags=4.0、summary=2.0、outcome=1.5、owner=1.0、detail=1.0

實作位置：src/lib/historySearch.ts ■ buildIndex(), searchHistory()

4.2 Robertson-Sparck-Jones IDF — 詞權重

```
idf(t) =  $\log(1 + (N - \text{df}(t) + 0.5) / (\text{df}(t) + 0.5))$ 
```

```
N =
```

```
df(t) = t
```

```
+0.5 = Lidstone smoothing
```

比 $\log(N/\text{df})$ 更穩健，stop word (的、了、是) 會自動接近 0 分，罕見詞 (東京中央銀行) 獲得高分。

4.3 多 n-gram 中文 Tokenization

```
tokenize(''):
```

```
1-gram: , , , , ,
```

```
2-gram: , , , ,
```

```
3-gram: , , ,
```

中文無詞界，1+2+3-gram 混合切。投資領域多為 3-4 字術語 (投委會、董事會、伊勢島飯店)，3-gram 特別重要。

4.4 Substring Boost

```
if document.toLowerCase().includes(query) → score × 1.8
```

```
if title.toLowerCase().includes(query) → score × 1.4
```

解決 n-gram 把專有名詞拆碎後被「銀行」這種 high-DF 詞稀釋的問題。完整匹配優先級高。

4.5 同義詞表 (14 組)

```
 ≈ ≈ ≈ fundraising
```

```
 ≈ ≈ DD ≈ due diligence
```

```
NDA ≈
```

```
LOI ≈
```

```
 ≈ valuation ≈
```

```
 ≈ exit ≈
```

```
 ≈ ≈ IC
```

```
 ≈ board
```

```
A ■ ≈ A■ ≈ series a
Pre-A ≈ PreA ≈ ■■■■
■■ ≈ compliance ≈ ■■
■■ ≈ tax
■■ ≈ ■■■■ ≈ risk
■■ ≈ client ≈ customer
```

Tokenize 前統一替換為 canonical form。從長到短替換避免子字串覆蓋（先「盡職調查」再「盡調」）。

4.6 Cosine Similarity（已退役）

$$\text{sim}(\mathbf{q}, \mathbf{d}) = (\mathbf{q} \cdot \mathbf{d}) / (||\mathbf{q}|| \times ||\mathbf{d}||)$$

v1 使用 TF-IDF + Cosine，因「線性 TF 計分」與「無欄位權重」兩個缺陷在 v2 被 BM25F 取代。本演算法不再用於主流程但保留說明，作為對照組。

6. ③ 時間序列（4 項）

6.1 Exponential Time Decay（指數時間衰減）

```
TIME_DECAY = [1.0, 0.7, 0.5, 0.35, 0.25, 0.15, 0.1, 0.05, 0.02]
              ↑ ↑ ↑ ↑ ↑ ...
              ■■ 1■■ 2■■ 3■■ 4■■
```

```
weeksAgo = round((asOf - reportWeek) / 7 days)
if weeksAgo >= 9: weight = 0 // ■■■■■■■■
else: weight = TIME_DECAY[weeksAgo]
```

用於：員工負載分數的時間加權、cross-dept network 的時間範圍切片。決策依據：本週案件實質權重 1.0，4 週前剩 25%，9 週前完全忽略，符合「近期工作影響更大」的人類直覺。

6.2 asOf Snapshot Computation — v2.1 重大重構

```
analyzeEmployeeLoad(reports, handoffs, employees, asOf: Date = NOW)
analyzeBlockerRecord(blocker, historyDB, history, asOf: Date = NOW)
computeHealthSnapshot(asOf, reports, handoffs, decisions, blockers, ...)
isDecisionOverdueAt(d, asOf: Date = NOW)
isDecisionCompletedAt(d, asOf: Date = NOW)

■■■■■■■■reportDate > asOf■■■■■■
```

為什麼 asOf 重要

原本所有分析器內部用 `new Date()` → 12 週趨勢線每週數字相同（毫無資訊價值）。v2.1 新增 asOf 參數讓「歷史快照」反映該週時點的真實狀態，趨勢線從此有意義。

6.3 Local Minima Detection（拐點偵測）

```
function detectInflectionPoints(series):
  result = []
  for i in 1..n-2:
    if series[i].overall < series[i-1].overall - 3
      AND series[i].overall < series[i+1].overall - 3:
      result.push(i)
  return result
```

閾值 3 分避免雜訊，只標真正顯著的 V 型谷底。在 12 週健康度趨勢圖上用紅點顯示。點選後展開該週事件清單。

6.4 Weekly Series Computation（12 週快照序列）

```
function computeWeeklySeries(weeks: number = 12):
  out = []
  for i in weeks-1 downto 0:
    asOf_i = NOW - i × 7 days
    out.push(computeHealthSnapshot(asOf_i, ...))
  return out // ■■ 12■■■■■■
```

對過去 12 週分別跑健康度快照。複雜度 $O(12 \times N)$ ， N = 資料總量。在組織健康度卡片繪製趨勢線。

7. ④ 加權評分模型（5 項）

7.1 Weighted Load Score（員工負載）

```
loadScore = timeWeightedCases × 1.5
            + blockerLoad × 2.0
            + mentionsWeighted × 0.8
            + handoffLoad × 1.0
```

Level ■■■■

```
overload = loadScore ≥ 25 OR percentile ≥ 90
high = loadScore ≥ 15 OR percentile ≥ 75
normal = loadScore ≥ 6
low = loadScore ≥ 1
idle = loadScore < 1
```

各分量定義：

timeWeightedCases	員工的案件複雜度 × decay。複雜度 = 含「請/需/協助/跨部門」（×1.5）或「卡/延/未通/缺漏/逾期」（×2.0）等關鍵字權重。
blockerLoad	員工負責的活躍卡點數 × 2.5 × decay（每筆卡點按 bullet 點數計）。外層再 ×2.0，所以一筆卡點實質權重 ×5.0。
mentionsWeighted	員工在他人週報中被提及的次數 × 1.5 × decay。外層再 ×0.8，避免「常被提到但實際沒分配工作」者被高估。
handoffLoad	員工作為交接接收者：待簽收 = 4 × decay、已簽收 = 1.5 × decay。外層 ×1.0。

7.2 ORI — Organizational Risk Index

```
ORI = 0.35 × HCC + 0.25 × DL + 0.25 × BT + 0.15 × CDC
```

```
HCC = clamp(100 + (Gini - 0.35) × 400 + (top1 - 0.2) × 200 + outliers × 8, 0, 200)
DL = clamp(100 + (avgCompletionDays - 14) × 4 + overdueCount × 12, 0, 200)
BT = clamp(100 + (avgP - 50) × 1.5 + p90 × 8 + p95 × 12, 0, 200)
CDC = clamp(100 + asymCount × 18 + asymRatio × 0.5, 0, 200)
```

■■ 0-200■■■■■■■■■■

五級告警：

ORI ≥ 175	今天要花時間 — 整週都會被拖住
ORI ≥ 150	要注意 — 有事在惡化
ORI ≥ 125	可關注 — 少量需注意
ORI ≥ 100	還可以 — 整體穩定
ORI < 100	順利 — 公司運作正常

7.3 Organization Health Score（6 維雷達） — v2.1 主指標

```
overall = blockerHealth × 0.22
            + decisionTimeliness × 0.18
            + handoffSmoothness × 0.15
```

```

+ loadBalance × 0.18
+ crossDept × 0.12
+ reportQuality × 0.15

```

■■ 0-100■■■■■■■■■■

六個維度子公式：

卡點健康 (22%)	$100 - P95\text{數} \times 15 - P90\text{數} \times 7 - \max(0, \text{avgPercentile} - 50) \times 0.8$
決策及時 (18%)	$100 - \max(0, \text{avgCompletionDays} - 14) \times 3 - \text{overdueCount} \times 10$
交接流暢 (15%)	$50 + \text{completionRate} \times 50 - \text{overdueHandoffs} \times 8$
負載均衡 (18%)	$100 - \max(0, \text{Gini} - 0.35) \times 200 - \text{overloadCount} \times 8$
部門協作 (12%)	$100 - \text{asymCount} \times 15$
週報品質 (15%)	$\min(1, \text{submitRate}) \times 60 + \text{lengthScore} \times 0.3 + \text{blockerFillRate} \times 10$

五級評等：優異 (≥ 85) / 良好 (≥ 70) / 可關注 (≥ 55) / 需注意 (≥ 40) / 亟需介入 (< 40)

7.4 Decision Impact Score（決策成效）— v2.1 新模型

```

before = computeHealthSnapshot(decidedAt - 1 day)
afterWanted = completedAt + windowWeeks × 7 // windowWeeks = 4
afterAsOf = min(afterWanted, NOW) // clamp ■■■■■■
after = computeHealthSnapshot(afterAsOf)

deltaOverall = after.overall - before.overall
score = deltaOverall
for each dimension:
  dimDelta = after.dim - before.dim
  if dimDelta ≥ 3: score += 2
  if dimDelta ≤ -3: score -= 2
score = clamp(score, -100, +100)

verdict = score ≥ +3 → '■■■'
≤ -3 → '■■■'
else → '■■■'

■ afterWanted > NOW: insufficient = true■■■■■ ■■■■■■■■■■

```

已知設計限制

本算法假設「決策後的健康度變化 = 該決策的成效」，但實際上組織健康度受多重因素影響。若整體趨勢正在下滑，所有決策的 delta 都會偏負（不是它們的鍋）。建議搭配 Cohort Adjustment 或 Dimension-specific Impact 改進。

7.5 Leader Scorecard（主管成效排行）

```

function computeLeaderScores(data):
  groups = groupBy(data.decisions, d => d.decidedBy)
  for each (decidedBy, decisions) in groups:
    completed = filter completedAt ≤ NOW

```

```
impacts = completed.map(d => analyzeDecisionImpact(d, data))
avgImpactScore = mean(impacts.map(i => i.score))
positiveCount = impacts.filter(i => i.verdict === '■■').length
...
sort by avgImpactScore desc
```

8. ⑤ 圖論 / 網絡分析 (3 項)

8.1 Directed Weighted Adjacency Matrix

```
matrix: Record<dept, Record<dept, number>> = {}

// ■■■■■■ mention
for each report r:
  text = r.cases + r.blockers + r.needHelp + r.nextWeek
  for each target ≠ r.dept:
    count = number of target mentions in text
    matrix[r.dept][target] += count

// ■■■■■
for each handoff h:
  matrix[h.from][h.to] += 1
```

實作位置：src/lib/algorithms.ts ■ analyzeDeptNetwork()

8.2 Force-directed Graph Layout

```
■■■■■ (Coulomb-like): F_rep = k_rep / distance2
■■■■■ (Hooke spring): F_attr = k_attr × distance × edge_weight
■■■:
  for step in 0..max_iter:
    for each node:
      net_force = Σ F_rep + Σ F_attr
      velocity += net_force × dt
      position += velocity × dt × damping
```

經典 D3.js 風格 force simulation。在 SVG

中以動畫迭代到平衡。節點大小依「對外溝通總量」、邊粗細依協作密度。

8.3 Asymmetric Communication Detection (單向溝通偵測)

```
asymCount = 0
for each (A, B) in depts × depts where A ≠ B:
  ab = matrix[A][B] || 0
  ba = matrix[B][A] || 0
  if ab ≥ 5 AND ba = 0:
    asymCount += 1
  asymRatio += ab
```

識別「A 部門一直找 B, B 完全沒回應 A」這種組織病徵。每組扣 15-18 分到 ORI 的 CDC 維度與健康度的部門協作維度。

9. ⑥ 狀態判定 (2 項)

9.1 Decision Status Helpers — v2.1 新增

```
function isDecisionOverdueAt(d, asOf = NOW): boolean
  if !d.dueDate || d.dueDate === '■■■■': return false
  due = new Date(d.dueDate)
  if isNaN(+due): return false
  if +due >= +asOf: return false // ■■■■■■
  if d.completedAt:
    completed = new Date(d.completedAt)
    if !isNaN(+completed) && +completed <= +asOf: return false
  return true

function isDecisionInProgressAt(d, asOf):
  if !d.decidedAt || new Date(d.decidedAt) > asOf: return false
  if d.completedAt && new Date(d.completedAt) <= asOf: return false
  return !isDecisionOverdueAt(d, asOf)

function isDecisionCompletedAt(d, asOf):
  if !d.completedAt: return false
  completed = new Date(d.completedAt)
  return !isNaN(+completed) && +completed <= +asOf

function daysOverdue(d, asOf):
  if !isDecisionOverdueAt(d, asOf): return 0
  return max(0, round((+asOf - +new Date(d.dueDate)) / 86400000))
```

為什麼需要這套 helper

v2.0 各頁面用 d.status === '逾期' 字串判斷，但 orgHealth 用動態日期判斷 → 兩套標準並存。v2.1 統一改用 helper，全系統一致，且自動偵測「dueDate 已過但 status 仍是執行中」的決策。

9.2 Risk / Load / Health Level — 多層級分類

Blocker Level (卡點等級)：

```
critical (■■■■) = currentDays ≥ P95 of ■■■■
high (■■■■) = currentDays ≥ P90
medium (■■■■) = currentDays ≥ P75
normal (■■■) = otherwise
```

Employee Load Level：

```
overload (■■) = loadScore ≥ 25 OR percentile ≥ 90
high (■) = loadScore ≥ 15 OR percentile ≥ 75
normal (■■) = loadScore ≥ 6
low (■) = loadScore ≥ 1
idle (■■) = loadScore < 1
```

Health Level：

```
■■ = overall ≥ 85
■■ = overall ≥ 70
■■■ = overall ≥ 55
■■■ = overall ≥ 40
■■■■ = overall < 40
```

10. ⑦ 預測 / 模擬（2 項）

10.1 What-if Scenario Simulation — v2.1 核心新功能

```
// State
scenario = {
  resolvedBlockerIds: Set<string>, // ■■■■■■
  expeditedDecisionIds: Set<string>, // ■■■■■■■■
  signedHandoffIds: Set<string>, // ■■■■■■■■
  extraHeadcount: { [dept]: 0..5 } // ■■■■
}

// ■■ scenario ■■ shadow data
function applyScenario(data, scenario):
  newBlockers = data.blockers.map(b => scenario.resolvedBlockerIds.has(b.id) ?
  {...b, status: 'resolved'} : b)
  newDecisions = data.decisions.map(d => scenario.expeditedDecisionIds.has(d.id) ?
  {...d, status: '■■■', completedAt: NOW} : d)
  newHandoffs = data.handoffs.map(h => scenario.signedHandoffIds.has(h.id) ? {...h,
  status: '■■■', hoursOverdue: undefined} : h)
  newEmployees = [...data.employees, ...generate(scenario.extraHeadcount)]

// ■■
baseline = computeHealthSnapshot(NOW, originalData)
projected = computeHealthSnapshot(NOW, applyScenario(data, scenario))
delta = projected.overall - baseline.overall
deltaByDimension = projected.dim - baseline.dim for each dim
```

實作位置：src/pages/WhatIf.tsx

10.2 Smart Suggestion（智能建議文案）

```
if delta ≥ +5 → '■■■■ ■■■■■■'
if delta in (+2, +5) → '■■■■■■■■■'
if delta in (-2, +2) → '■■■■■■■■■■'
if delta in (-5, -2) → '■■■■■■■■■'
if delta ≤ -5 → '■■■■ ■■ ■■■■■'
```

把 delta 數字轉成人類可讀的決策建議，降低管理層的解讀成本。搭配雙圖層雷達（灰色 = 現況、紫色 = 模擬後）視覺化。

12. 全域常數定義表

12.1 演算法常數

k1 (BM25)	1.5 — TF 飽和速度。建議範圍 1.2-2.0。
b (BM25)	0.75 — 長度正規化強度。0=不正規化、1=完全正規化。
FIELD_WEIGHTS	title=5.0、tags=4.0、summary=2.0、outcome=1.5、owner=1.0、detail=1.0
TIME_DECAY	[1.0, 0.7, 0.5, 0.35, 0.25, 0.15, 0.1, 0.05, 0.02] (9 週)
Gini 公平/不公平分界	0.35 (學術標準)
Local Minima 閾值	3 分 (拐點顯著性)
Decision Impact windowWeeks	4 週
Org Health 權重	卡點 22% / 決策 18% / 交接 15% / 負載 18% / 協作 12% / 週報 15%
ORI 權重	HCC 35% / DL 25% / BT 25% / CDC 15%
Load Score 權重	cases 1.5 / blocker 2.0 / mentions 0.8 / handoff 1.0

12.2 等級閾值

Blocker critical	currentDays $\geq$ P95
Blocker high	currentDays $\geq$ P90
Blocker medium	currentDays $\geq$ P75
Load overload	loadScore $\geq$ 25 OR percentile $\geq$ 90
Load high	loadScore $\geq$ 15 OR percentile $\geq$ 75
Health 優異	overall $\geq$ 85
Health 良好	overall $\geq$ 70
Health 可關注	overall $\geq$ 55
Decision Impact 正面	score $\geq$ +3
Decision Impact 負面	score $\leq$ -3
Handoff overdue 24h	hoursOverdue $\geq$ 24 計入 critical
Asymmetric 偵測	weight(A $\rightarrow$ B) $\geq$ 5 AND weight(B $\rightarrow$ A) = 0

12.3 SEED 資料量

SEED_EMPLOYEES	20 位 (管理層 3 + 投研 8 + 業開 6 + 資管 4)
SEED_REPORTS	~240 筆 (80 週 $\times$ 3 部門 + 本週 3 筆手寫)
SEED_HANDOFFS	~150 筆 (80 週, 每週 1-2 筆)

SEED_HISTORY	53 筆（按 6 大類別分布）
SEED_DECISIONS	18 筆（7 執行中 + 4 逾期 + 7 已完成）
SEED_BLOCKERS	12 筆（2 極高 + 3 高 + 3 中 + 4 正常）
SEED_MEETING_HISTORY	12 場（週會 9 + 月會 3）
caseRegistry (公司)	26 家池井戶潤宇宙公司（金融、製造、IT、飯店、航空、建設）

13. SEED 資料結構

13.1 真實企業池 (caseRegistry)

依產業分類的 26 家虛構但具沉浸感的公司：

[illegible]

13.2 員工池（依部門）

營運與管理層 (3)	吳君(董事長)、陳文翰(COO)、黃詩涵(CFO)
投資研究部 (7)	周世倫、鍾皓明、張偉、李有廷、謝佳穎、王子翔、廖宜萱
業務開發部 (6)	林聿平、林欣逸、蔡明遠、楊雅雯、羅宇晴、陳俊宏
資產管理部 (4)	梁嘉芫、陳雅文、蘇柏豪、邱筱慧

13.3 卡點類別 (6 大類)

法遵/合約	rose 色 · 平均 7-8 天 · 對應東京中央銀行、白水銀行等金融標的
資金/募資	amber 色 · 平均 5-6 天 · 對應飯店、航空、製造類
資料/補件	sky 色 · 平均 6-7 天 · 對應製造業 DD
跨部門/窗口	violet 色 · 平均 4-5 天 · 對應 IT 類
決策/簽核	indigo 色 · 平均 8-10 天 · 對應大型銀行
時程/聯繫	emerald 色 · 平均 5-6 天 · 對應飯店、航空

14. 色彩系統與 UI 等級配色

14.1 健康度配色

優異 (≥85)	emerald-600 / emerald-500 / emerald-50
良好 (≥70)	blue-600 / blue-500 / blue-50
可關注 (≥55)	amber-600 / amber-500 / amber-50
需注意 (≥40)	orange-600 / orange-500 / orange-50
亟需介入 (<40)	red-600 / red-500 / red-50

14.2 卡點風險配色

critical (極高)	red-500 + 紅色 gradient
high (高)	amber-500 + 琥珀 gradient
medium (關注)	blue-500
normal (正常)	emerald-500

14.3 部門配色

投資研究部	blue (藍 — 研究、分析)
業務開發部	emerald (綠 — 開發、成長)
資產管理部	violet (紫 — 風險、長期)
營運與管理層	slate (灰 — 中性)

14.4 速度標籤配色（歷史搜尋）

快速解決 (≤3 天)	emerald-50 / emerald-600
正常解決 (4-7 天)	blue-50 / blue-600
較慢解決 (8-14 天)	amber-50 / amber-600
嚴重延誤 (>14 天)	rose-50 / rose-600

15. 11 個主要頁面與功能矩陣

Dashboard	管理層摘要儀表板	整體狀態 + 3 Hero Cards + 組織健康度 + 今日 3 件事
WeeklyReport	週報填寫 + 歷史	本週狀態 3 卡 + 填寫表單 + 歷史 12 週累積
Handoff	案件交接	2 大狀態板 + 列表 + Modal 詳情 + 智能推薦
Decisions	決策追蹤	3 大狀態板 + 主管成效排行 + Decision Impact 詳情
EmployeeLoad	員工負載 + 週次切換	12 週切換器 + 過載/閒置篩選 + 4 元素 breakdown
History	歷史案件搜尋	BM25F 搜尋 + 速度標籤 + 類別 chip + 詳情 Modal
BlockerAnalytics	卡點分析	4 風險磚 + 點擊展開卡點清單 + 類別分布
OrgAnalytics	部門互動網絡	Force-directed SVG + 時間範圍 + 節點點擊
MeetingPrep	會議準備	3 卡片視覺議程 + 歷史會議分類
WhatIf	決策模擬器 (v2.1)	4 情境模組 + 雙圖層雷達 + 智能建議
LineBot	LINE 推播設定	推播規則 + 訂閱者 + 範本

16. 權限模型與角色

16.1 三層角色

admin (管理員)	看全部 11 個分頁。董事長、COO、CFO
manager (主管)	看 8 個分頁（不含員工負載、組織分析、會議準備）。各部門主管
member (一般員工)	看 4 個分頁（Dashboard、週報、交接、LineBot）

16.2 角色推斷邏輯

```
function inferUserProfile(email): UserProfile
  ■ users collection ■ email ■■■■
  if ■■ → ■■ SystemUser ■ role ■■
  else → fallback ■ 'member'
  ■■ displayName, dept ■■■■
```

16.3 頁面權限檢查

```
// App.tsx
if (navItem && !navItem.roles.includes(userRole)) {
  setActiveTab('dashboard') // ■■■■■■
}

// Sidebar.tsx
const items = NAV.filter(n => n.roles.includes(role || 'member'))
```

17. 通知中心與事件流

17.1 通知來源（4 大類）

```
function buildNotifications(data):  
  list = []  
  
  // 1. P95+ / P90+ ■■■■■  
  blockers.filter(level in ['critical', 'high']).top(5)  
    → list.push({ level: 'critical'/'warn', tab: 'analytics' })  
  
  // 2. ■■■■■  
  decisions.filter(isDecisionOverdueAt).top(5)  
    → list.push({ level: 'critical', tab: 'decisions' })  
  
  // 3. ■■■■ + ■■ 24h ■■  
  handoffs.filter(status='■■■■' && hoursOverdue > 0).top(5)  
    → list.push({ level: (≥48h ? 'critical' : 'warn'), tab: 'handoff' })  
  
  // 4. ■■■■■■■■■■  
  activeDepts.filter(!submitted)  
    → list.push({ level: 'info', tab: 'report' })  
  
  return list
```

17.2 視覺呈現

徽章顏色	紅(有 critical) / 黃(僅 warn) / 無(空)
徽章數字	totalCount, 超過 9 顯示 '9+'
展開行為	點擊鈴鐺 → 滑出 96 寬下拉、點外面關閉、點 X 關閉
項目互動	點任一項 → onNavigate(tab) 跳對應頁面 + 關閉面板
空狀態	顯示 ✓ 圖示 + 「一切順利」

18. Bug 修復記錄 (v2.0 → v2.1)

agent 深度掃描報告找出的 9 件 bug，於 commit 5a0cfd4 一次修復。涵蓋跨頁面數據一致性、邊界條件、SEED 保護。

B1	確定	Dashboard analyzeBlockerRecord 漏傳 history percentile / level 與其他頁不一致	補上第 3 個參數 history
B5	確定	analyzeBlockerRecord 用 new Date() 算 currentDays 歷史快照 days 永遠偏大、12 週趨勢不準	新增 asOf 參數，預設 NOW；orgHealth + OrgHealthCard 都傳入
B2	確定	d.status === '逾期' 字串 vs orgHealth 動態判定 兩套標準並存，用戶看到健康度被扣分卻找不到對應決策	新增 4 helper (isDecisionOverdueAt 等)，全系統統一改用
B3	確定	dueDate 為 'NaN 字串' 或 '即時生效' 畫面顯示「逾期 NaN 天」	helper 內部 isNaN 檢查 + clamp to 0
B6	確定	decisionImpact.afterAsOf 落在未來 「決策後快照 = 現況快照」，無法區分前後影響	clamp 到 NOW，未滿 4 週標記「追蹤中」
B8	潛在	orgHealth submissionRate 可超過 1 管理層交週報導導致分子大於分母、reportQuality 異常高	分母與分子皆過濾 expectedDeptSet，加 Math.min(1, ...)
B9	潛在	EmployeeLoad 過載百分比在空資料 顯示 'NaN%'	加 loads.length > 0 守門
B7	潛在	Firestore departments / users 集合誤刪 setDepartments([]) → 整個 app 崩潰	兩個集合都加 SEED fallback
B11	建議	Dashboard 未使用的 Sparkles import TS 雖然通過但程式碼整潔度差	移除

19. 已知設計限制與未來改進方向

19.1 Decision Impact 的因果歸因問題

本算法假設「決策後的健康度變化 =

該決策的成效」。實際上組織健康度受多重因素影響：若整體趨勢正在下滑（如目前 SEED 中 5 週前到本週掉了 18 分），所有完成於此期間的決策，其 delta 都會偏負，不是它們的鍋。

改進方向：

(A) Cohort Adjustment：每筆決策的 delta 扣掉同期基準變化。 $\text{adjusted_delta} = \text{decision_delta} - \text{baseline_delta_of_period}$ 。

(B) Window Shortening：4 週 → 1 週，減少其他事件干擾。

(C) Dimension-specific Impact：只比較決策直接相關的維度。

(D) Causal Inference：加 propensity score matching 或更複雜的 attribution model。

19.2 歷史快照的 percentile 偏差

analyzeBlockerRecord 現已接受 asOf，但 stats.percentile 是對「現在的歷史資料」算分位。若想完全還原 5 週前的 percentile，需要對該週的「同類歷史 + asOf 之前已解決」過濾。目前簡化為用全部 history（53 筆）算 percentile，誤差不大但非完美。

19.3 Force-directed Layout 收斂

目前 SVG 實作為簡化版（無 quadtree、無 alpha decay），對於 7 個部門節點足夠，但部門數 > 20 時會明顯卡頓。改進方向：(A) 用 D3 force simulation 套件 (B) Worker 卸載計算。

19.4 What-if 加員工的 loadBalance 影響

目前模擬員工 loadScore = 0，Gini 反而可能被「全 0 員工」拉高（不均度增加）。改進方向：模擬時把 overload 員工的部分分數平移到新員工（reassignment 機制）。

19.5 BM25F 對純英文查詢

目前 n-gram 主要針對中文（一-齣 Unicode range）。英文用 \w+ 整詞匹配，無 stemming（如 "investing" 與 "investment" 不互通）。改進方向：加 Porter Stemmer 或 Snowball。

20. 演算法選用對照與設計哲學

20.1 對照表

資訊檢索	BM25F (Lucene 同款) — 而非 TF-IDF cosine 或 LLM Embedding
風險量化	Empirical Percentile — 而非絕對天數門檻
不平均度	Gini 係數 (經濟學標準) — 而非標準差
時間衰減	Exponential Decay 含 asOf 切片 — 支援歷史快照
拐點偵測	Local Minima — 而非機器學習 anomaly detection
組織網絡	Force-directed Graph + 雙向 Matrix
決策狀態	動態日期判定 helper — 不依賴可能過期的 status 字串
決策成效	前後快照對比 + 時間 clamp 避免未來
文字搜尋	純前端 BM25F — 不送 cloud API

20.2 為什麼不用 LLM?

- (1) 資料量小：53 筆歷史案、240 筆週報、150 筆交接。BM25F 比 Embedding 更可靠且零成本。
- (2) 可解釋性：管理層需要知道「為什麼推薦這筆」，BM25F 可逐項列出命中詞貢獻。
- (3) 機密敏感：投資公司資料不能送 cloud API，本系統全部前端 + 自有 Firebase。
- (4) 計算成本：BM25F、Gini、percentile 都是 $O(N)$ 或 $O(N \log N)$ ，瀏覽器毫秒級。
- (5) 確定性：相同輸入永遠回傳相同結果，LLM 有 stochasticity。

20.3 Plan → Decide → Track → Learn 閉環

v2.1 形成完整的決策週期：

- Plan (規劃) · 卡點分析 + 員工負載 + 組織健康度 看現狀
- Decide (決策) · What-if 模擬器 決策前看後果
- Track (執行) · 決策追蹤 + 交接 + 週報
- Learn (學習) · Decision Impact 量化效益、Leader 排行

下次做決策時，系統可用過去的 impact 推薦類似類型 — 形成自我強化的決策智慧。

附錄 A. 演算法快速索引（25 項對應檔案）

#	演算法	類別	檔案 + 函式
1	BM25F	資訊檢索	historySearch.ts ■ searchHistory()
2	Robertson-Sparck-Jones IDF	資訊檢索	historySearch.ts ■ buildIndex()
3	多 n-gram Tokenization	資訊檢索	historySearch.ts ■ tokenize()
4	Substring Boost	資訊檢索	historySearch.ts ■ searchHistory()
5	Synonym Normalization	資訊檢索	historySearch.ts ■ SYNONYM_MAP
6	Cosine Similarity (退役)	資訊檢索	v1 ■■
7	Empirical Percentile	統計	algorithms.ts ■ stats.percentile()
8	Gini Coefficient	統計	algorithms.ts + orgHealth.ts
9	Mean / Std / Median	統計	algorithms.ts ■ stats
10	Exponential Time Decay	時間序列	algorithms.ts ■ getDecayWeight
11	asOf Snapshot	時間序列	algorithms.ts + orgHealth.ts
12	Local Minima Detection	時間序列	orgHealth.ts ■ detectInflectionPoints()
13	Weekly Series	時間序列	orgHealth.ts ■ computeWeeklySeries()
14	Weighted Load Score	加權評分	algorithms.ts ■ analyzeEmployeeLoad()
15	ORI Index	加權評分	algorithms.ts ■ computeORI()
16	Org Health 6D Score	加權評分	orgHealth.ts ■ computeHealthSnapshot()
17	Decision Impact Score	加權評分	decisionImpact.ts ■ analyzeDecisionImpact()
18	Leader Scorecard	加權評分	decisionImpact.ts ■ computeLeaderScores()
19	Adjacency Matrix	圖論	algorithms.ts ■ analyzeDeptNetwork()
20	Force-directed Layout	圖論	OrgAnalytics.tsx
21	Asymmetric Detection	圖論	algorithms.ts + orgHealth.ts
22	Decision Status Helpers	狀態判定	algorithms.ts ■ isDecisionOverdueAt ■
23	Risk / Load / Health Level	狀態判定	■■
24	What-if Scenario Simulation	預測模擬	WhatIf.tsx ■ applyScenario
25	Smart Suggestion	預測模擬	WhatIf.tsx

附錄 B. 公式速查表

BM25 TF	$(tf \times (k1+1)) / (tf + k1 \times (1 - b + b \times len/avgLen))$
RSJ IDF	$\log(1 + (N - df + 0.5) / (df + 0.5))$
Substring Boost	<code>doc.contains(q) → ×1.8 ; title.contains(q) → ×1.4</code>
Percentile	<code>sorted[(n-1) × p/100] (linear interp)</code>
Gini	$\Sigma (2i-n-1) \times s[i] / (n \times \Sigma s[i])$
Time Decay	<code>[1.0, 0.7, 0.5, 0.35, 0.25, 0.15, 0.1, 0.05, 0.02]</code>
Local Minima	<code>s[i] < s[i-1] - 3 AND s[i] < s[i+1] - 3</code>
Load Score	<code>cases×1.5 + blocker×2.0 + mentions×0.8 + handoff×1.0</code>
ORI	<code>0.35×HCC + 0.25×DL + 0.25×BT + 0.15×CDC</code>
Org Health	<code>0.22×blocker + 0.18×decision + 0.15×handoff + 0.18×load + 0.12×cross + 0.15×report</code>
Decision Impact	<code>deltaOverall + Σ(dim ±2)■clamp [-100, +100]</code>
Overdue Check	<code>dueDate < asOf AND (!completedAt OR completedAt > asOf)</code>
Days Overdue	<code>max(0, round((asOf - dueDate) / 86400000))</code>
Asym Comm	<code>weight(A→B) ≥ 5 AND weight(B→A) = 0</code>
Health Level	<code>≥85■■■ / ≥70■■■ / ≥55■■■■ / ≥40■■■■ / <40■■■■■</code>

附錄 C. FAQ — 教授可能問的 12 個問題

Q1: 為什麼用 BM25F 不用 GPT Embedding?

資料量小（200 筆級別）、可解釋性、機密敏感、零成本、確定性。詳見 § 20.2。

Q2: 健康度分數怎麼算的?

6 維加權平均，公式見 § 7.3。每維度有獨立子公式（卡點 P95+ 扣 15、決策逾期扣 10...）。

Q3: 為什麼 Gini 0.35 是分界?

經濟學標準。國家層級 < 0.35 為高度平等社會、> 0.4 為高度不平等。組織內也適用。

Q4: 12 週趨勢線的拐點怎麼找的?

Local Minima Detection，閾值 3 分避免雜訊。詳見 § 6.3。

Q5: Decision Impact 全是負分，怎麼回事?

因果歸因限制，組織整體趨勢下滑期間的決策都會被「冤枉」。改進方向見 § 19.1。

Q6: 為什麼員工負載要用 percentile?

公司規模不同、產業淡旺季不同，固定門檻不適用。Percentile 自動適應分布。

Q7: 時間衰減為什麼選指數而非滑動視窗?

指數平滑漸進；滑動視窗有「邊緣斷裂」問題。本週案件不會 8 週後突然消失。

Q8: 健康度跟 ORI 有什麼不同?

兩者底層共享分析器。ORI 0-200 反向（內部 dashboard 用）、健康度 0-100 正向（管理層 demo）。

Q9: 部門協作維度怎麼算?

從週報文字 + 交接紀錄萃取部門 mention，建構有向圖。偵測單向溝通（ $A \rightarrow B \geq 5$ 、 $B \rightarrow A = 0$ ）扣分。

Q10: What-if 模擬器的數學基礎?

對原始資料 fork shadow data，套用修改後重跑 computeHealthSnapshot。雙圖層雷達對比 delta。

Q11: 為什麼決策有「執行中 / 逾期」兩個狀態但用 helper 動態判定?

v2.0 用 status 字串時，dueDate 過期但 status 未更新會錯。v2.1 改動態判定避免人為誤差。

Q12: 系統未來可以怎麼擴展?

(A) 卡點解決時間預測（多元迴歸）(B) 員工技能向量化 + 智能分派 (C) 投組即時行情看板 (D) 一鍵董事會 PDF (E) 語音輸入週報。詳見聊天歷程的「亮點建議」。

串連系統 v2.1 完整技術參考手冊

共 25 演算法 / 機制、11 個 UI 模組、10 個 collection、9 件 bug 修復記錄。

「資料越少，演算法的選擇越重要。」

「跨頁面的一致性，比單頁的炫技更重要。」

—— 串連系統設計哲學